A Decision Framework for Log Aggregation Strategy Selection in High-Duplication Kubernetes Workloads

N. P. Ponnamperuma
2020/ICT/97

Table of contents

Research Objectives and Scope

The Problem

When containers log repeated log lines in situations such as retry loops and error storms, the same message can be logged hundreds or thousands of times within seconds, differing only in the timestamp. This redundant transmission wastes network bandwidth, inflates storage costs, and degrades log search query performance.

General Objective

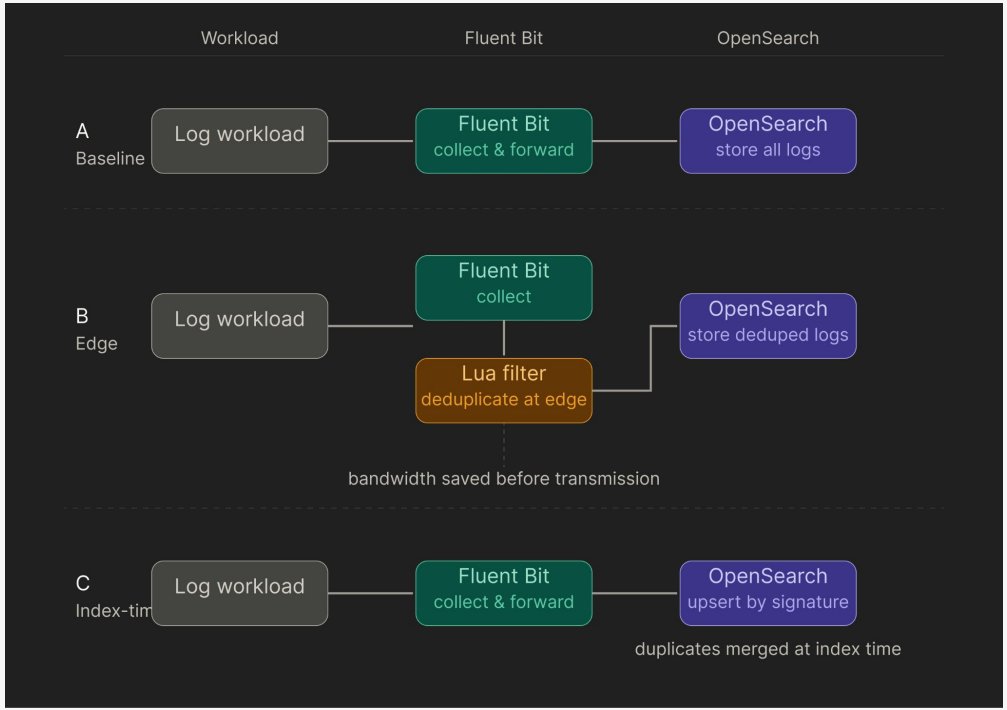
Create a decision framework for Devops engineers to determine the best trade-off between storage reduction and information preservation across logging pipeline configurations.

Scope

- **In scope:** Analyzing edge side and index-time log aggregation strategies for Kubernetes workloads.
- **Out of scope:** Any aggregation or deduplication occurring outside these two layers, including application-level and network-level approaches.

Research Method

The Three Pipelines



What we measure per run:

- Storage Reduction Ratio (SRR)
- Information Preservation Ratio (IPR)
- Compression Efficiency (CE)
- Failure Loss Rate (FLR) under simulated pod crash

Progress to Date

- Created infrastructure for the experiment: A k8s cluster running on DigitalOcean
- Designed the experiment to collect necessary data around the 3 pipelines. All pipelines and experiments are implemented and deployed on the k8s cluster.
- Experiment matrix scripted and automated
- Currently in experiment and data collection phase

Remaining Work

Experiment & Data Collection (Current Phase)

- Execute 300 experiment runs: 100 workloads x 3 pipelines (clean-slate mode)
- Execute additional 300 runs in no-clean-slate cumulative mode
- Execute failure scenario runs for each pipeline under selected workloads

Analysis

- Calculate SRR, IPR, CE, and FLR per run
- Generate trade-off curve, failure impact graph, category box plots, and cumulative storage growth graph
- Produce comparison table

Validate and discuss the effectiveness of the results and decision framework with domain experts from the industry.

Planned vs. Actual Progress

Workstream	Planned (per Gantt)	Actual	Status
Title selection	Nov 2025	Done	**On track**
Proposal	Dec 2025	Done	**On track**
Literature Review	Dec 25 - May 26	Ongoing	**On track**
Resource gathering	Jan - Feb 2026	Done	**On track**
Tools and Techniques	Jan - Feb 2026	Done	**On track**
Implementation	Feb - Apr 2026	Ongoing	**On track**
Documentation Writing	Jan - May 2026	Not started yet	**Behind**
Publication	May 2026	Due	*Upcoming*
Presentation & Submission	Jun 2026	Due	*Upcoming*

References

1. Singh, S. (2016). "Cluster-level Logging of Containers with Containers". ACM Queue, 14(3), 2-24.
2. Yao, Z. et al. (2021). "CLP: Efficient and Scalable Search on Compressed Text Logs". 15th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 183-196.
3. He, P. et al. (2017). "Drain: An Online Log Parsing Approach with Fixed Depth Tree". 2017 IEEE International Conference on Web Services (ICWS), 33-40.
4. Li, S. and Lan, T. (2022). "Pushing Collaborative Data Deduplication to the Network Edge". IEEE Transactions on Network Science and Engineering, 9(5), 3155-3171.
5. Usman, M. et al. (2022). "A Survey on Observability of Distributed Edge and Container-Based Microservices". IEEE Access, 10, 86904-86919.

Thank You!

What Changed and Why

Original proposal: Design and evaluate an edge-side semantic log aggregation system (including the prototype, efficiency metrics, and experiment).

Enhancement: The experiments remain, but a comparison across pipeline configurations is added to create a decision framework for DevOps engineers. By framing the original work as Pipeline B and adding Pipeline A (baseline) and Pipeline C (index-time), a comprehensive framework becomes possible.

Is the Decision Framework Novel?

What exists

- Compressed Log Processor (CLP) , SMT, Drain: backend compression and log parsing, all operating after ingestion
- Fluent Bit: basic filtering with no semantic awareness
- Li and Lan (2022): edge deduplication for general data, not log-specific patterns

What does not exist

- An empirical comparison of collector-layer vs. index-time aggregation for Kubernetes log workloads
- A practitioner-facing decision framework grounded in storage-information trade-off data at varying duplication ratios

The gap is real. No existing work gives a DevOps engineer a data-backed answer to: given my workload duplication profile, which aggregation strategy should I deploy?